Лабораторная работа №18
Изучение процесса применения обобщений

1 Цель работы

1.1 Изучить процесс разработки обобщенных функций и классов на C#;

1.2 Закрепить навык разработки программ методами структурного, процедурного и модульного программирования.

2 Литература

2.1 Ашарина, И. В. Объектно-ориентированное программирование в C++ : лекции и упражнения. Учебное пособие для вузов. / И. В. Ашарина. – 2-е изд., перераб. и доп. – Москва : Горячая Линия–Телеком, 2017. – 336 с. – URL: https://ibooks.ru/reading.php?productid=359752. – Режим доступа: для зарегистрир. пользователей. – Текст : электронный. – гл.25.

3 Подготовка к работе

3.1 Повторить теоретический материал (см. п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Разработать обобщенный метод `Max` для определения и возврата максимального значения из двух переданных в параметрах. Для корректной работы обобщения необходимо установить ограничение where T : IComparable<T> и сравнивать значения при помощи CompareTo

Протестировать метод, вызвав его со значениями различных типов данных (числа разных типов, строки). Значения задать программно в основной программе.

5.2 Создать обобщенный класс `Result` (Результат некоторой операции, который может завершиться успешно или с ошибкой):

```
class Result<TData, TError>
```

Добавить в класс следующие открытые элементы:

- свойство bool IsSuccess, которое показывает, успешно ли завершилась операция
- свойство Data, содержащее результат
- свойство Error, содержащее информацию об ошибке
- статический метод Success(TData data) для создания успешного результата
- статический метод Failure(TError error) для создания результата с ошибкой

5.3 Протестировать класс, создав метод

```
Result<int, string> Divide(int a, int b)
```

При делении на ноль возвращать Failure с текстовым сообщением об ошибке, при корректных данных возвращать Success с результатом вычисления.

Проверить метод в основной программе

5.4 * Разработать метод:

```
public static TResult[] Transform<TSource, TResult>(
    TSource[] source,
    Func<TSource, TResult> converter)
```

Метод должен преобразовывать элементы исходного массива по определенному правилу.

Метод должен создавать новый массив типа TResult[].

Каждый элемент исходного массива должен передаваться функции converter.

Результат преобразования должен записываться в соответствующую позицию нового массива

Пример использования:

```
int[] numbers = { 10, 20, 30 };

string[] result = Transform<int, string>(
    numbers,
    number => $"Число: {number}"
);
```

Массив result должен содержать следующие значения:
Число: 10
Число: 20
Число: 30

6 Порядок выполнения работы

6.1 Используя Microsoft Visual Studio, создать проект C# и выполнить задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Для чего применяются обобщения в C#?

8.2 Какова общая форма обобщенной функции?

8.3 Какова общая форма обобщенного класса?

8.4 Как передать в обобщение несколько типов?

9 Приложение

Обобщения (Generics) позволяют создавать методы и классы, которые работают с различными типами данных без необходимости дублирования программного кода.
Вместо конкретного типа данных используется параметр типа, который указывается в угловых скобках <T>.

Обобщённый метод содержит один или несколько параметров типов, определяемых при его вызове.
Например, метод для вывода значения любого типа:

```
static void PrintValue<T>(T value)
{
    Console.WriteLine(value);
}
Пример использования:
PrintValue<int>(10);
PrintValue<string>("Привет");
PrintValue<double>(3.14);

// Тип также может определяться автоматически
PrintValue(25);
```

Ограничения позволяют задавать требования к типам, которые могут использоваться при создании обобщённых объектов или вызове методов.

Ограничение	**Назначение**
where T : class	Ссылочный тип
where T : struct	Значимый тип, не допускающий null
where T : new()	Наличие публичного конструктора без параметров
where T : IComparable<T>	Реализация интерфейса сравнения
where T : BaseClass	Наследование от указанного класса

Обобщённый класс позволяет использовать параметры типов при объявлении полей, свойств и методов.
Класс Message<TSender, TContent>, представляющий сообщение, которое содержит информацию об отправителе и содержимом. Типы отправителя и содержимого могут быть различными.

```
public class Message<TSender, TContent>
{
    public TSender Sender { get; set; }
    public TContent Content { get; set; }

    public Message(TSender sender, TContent content)
    {
```

```
        Sender = sender;
        Content = content;
    }

    public void Print()
    {
        Console.WriteLine($"Отправитель: {Sender}");
        Console.WriteLine($"Содержимое: {Content}");
    }

    public void UpdateContent(TContent newContent)
    {
        Content = newContent;
    }
}
```

Класс содержит два параметра типа:

- TSender – определяет тип отправителя сообщения.
- TContent – определяет тип содержимого сообщения.

При создании объекта необходимо указать конкретные типы обоих параметров.

Использование класса

```
// Отправитель — строка, содержимое — строка
Message<string, string> message1 =
    new Message<string, string>("Иван", "Привет!");

message1.Print();

// Отправитель — число, содержимое — строка
Message<int, string> message2 =
    new Message<int, string>(101, "Задание выполнено");

message2.Print();

// Отправитель — строка, содержимое — число
Message<string, int> message3 =
    new Message<string, int>("Датчик температуры", 25);

message3.Print();

// Изменение содержимого
message3.UpdateContent(30);
message3.Print();
```